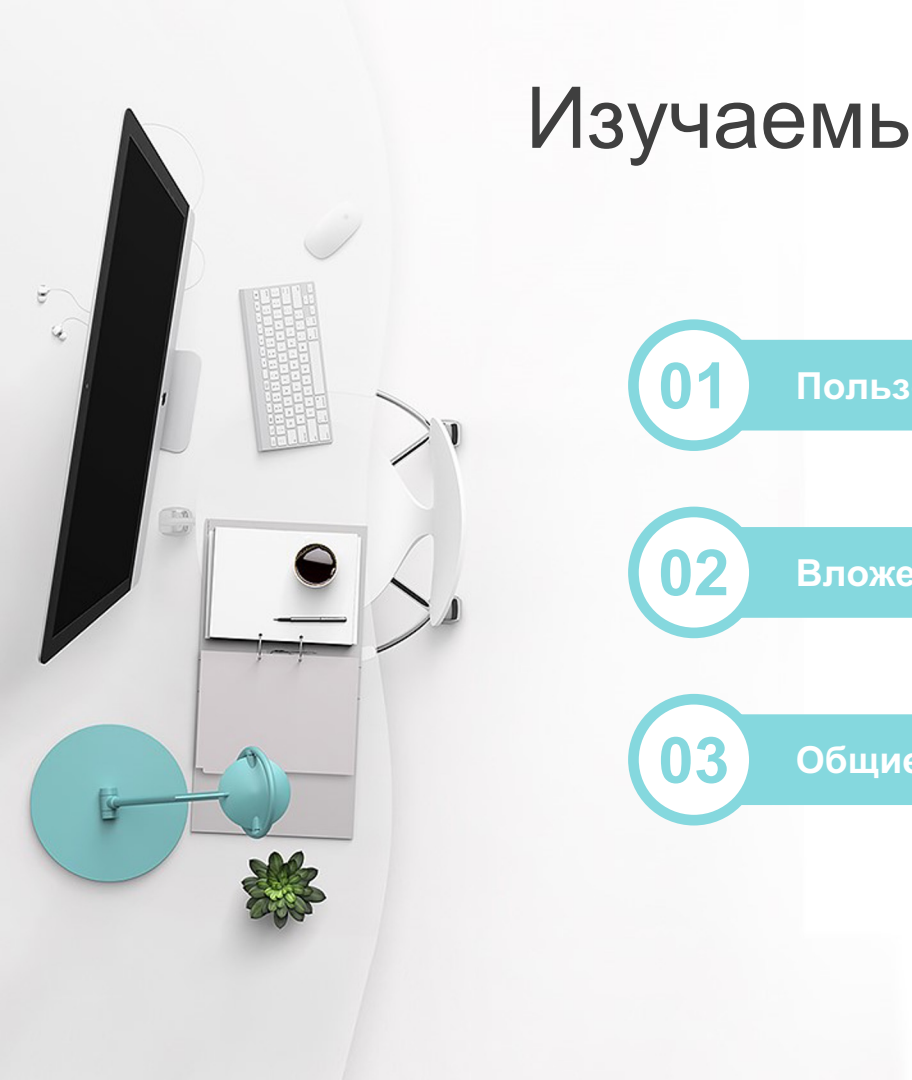
Базы данных

доктор технических наук, приглашенный преподаватель ВШБ
Стоянова Ольга Владимировна

Способы инкапсуляции бизнес-логики при анализе данных в БД

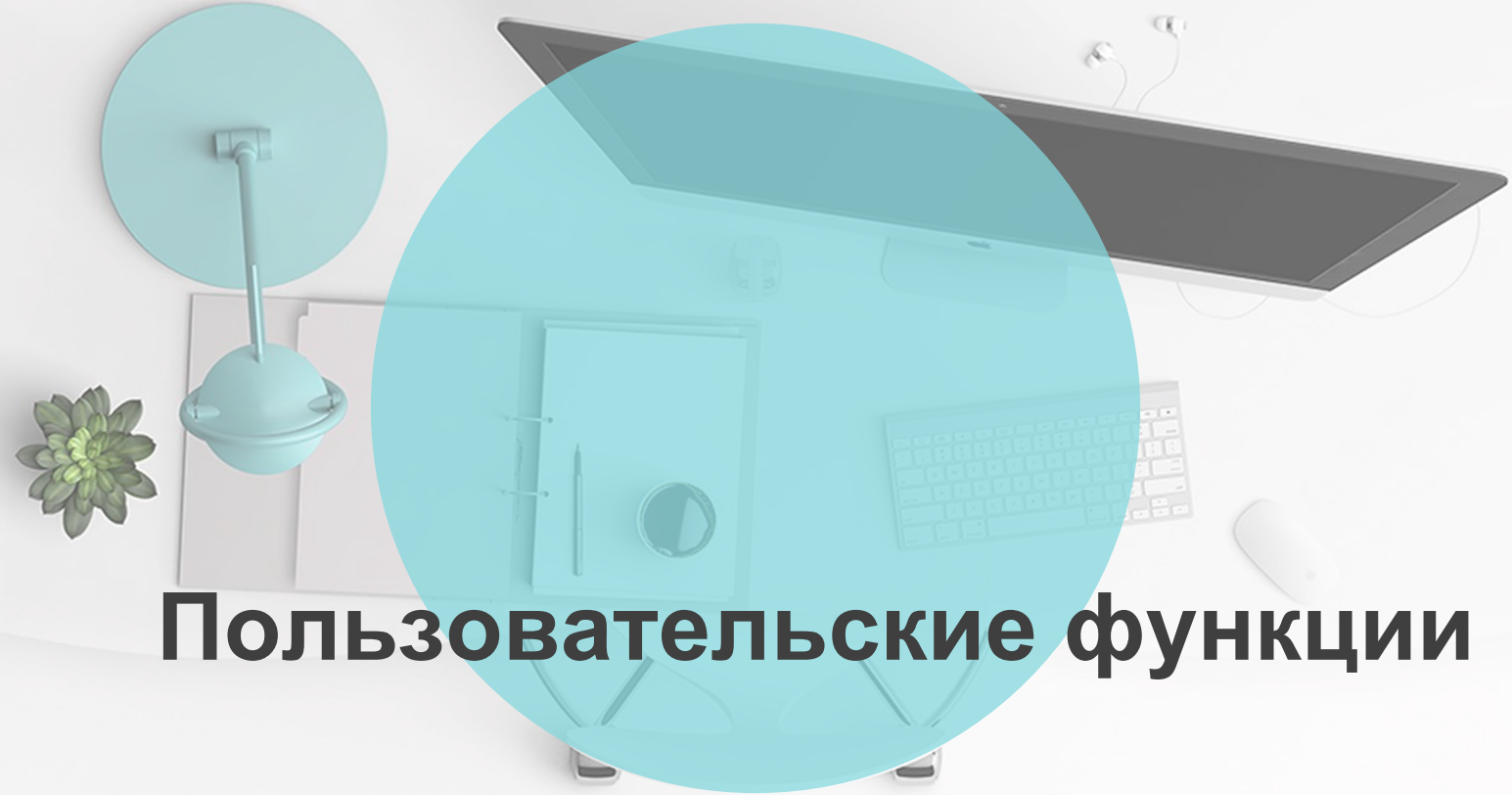


Изучаемые вопросы

01 Пользовательские функции

02 Вложенные подзапросы разных типов

03 Общие табличные выражения



Пользовательские функции

Понятие пользовательских процедур и функций

- Пользовательские функции
  - реализуют бизнес-логику по обработке данных
  - могут быть написаны для SQL, на процедурных языках PL/pgSQL или PL/Tcl, на C совместимых языках
  - могут принимать/возвращать данные
  - вызываются как часть запроса или команды DML

- Пользовательские процедуры
  - реализуют бизнес-логику по обработке данных
  - не возвращают данные
  - вызываются командой CALL

Создание функций (упрощенный вид объявления)

```
CREATE [ OR REPLACE ] FUNCTION имя ([[ имя_аргумента ]
тип_аргумента] [, ...] ] )
[RETURNS тип_результата |
RETURNS TABLE ( имя_столбца тип_столбца [, ...] ) ]
AS строка - тело функции
LANGUAGE имя_языка;
```

Тело функции

- Включает команды на выбранном языке
- Имеет строковый тип
- Интерпретируется при вызове

Варианты оформления

```
' Select …. ; '
$$ Select …. ; $$
$func$ Select …. ; $func$
```

Аргументы функции

```
[ режим_аргумента ]
[ имя_аргумента ]
тип_аргумента
[ { DEFAULT | = }
выражение_по_умолчанию ]
```

Возвращаемые значения

Конструкция RETURNS

- RETURNS ***тип_результата*** | ***setof*** |***table***
- RETURNS ***void***

Выходные аргументы

- режим OUT или INOUT

Пример пользовательской функции

```
--1.    Создайте функцию, возвращающую число сотрудников в компании.
create function my_f_1 () returns bigint
as
$$
select count(tab_nom) from employee;
$$
Language SQL;

select * from my_f_1 ();
```

Output Messages Notifications

my_f_1 bigint
10

Пример пользовательской функции

```
--2.Создайте функцию, возвращающую число сотрудников в компании и средний возраст сотрудников компании.
create or replace function empl_age_1 (out cnt bigint, out ag numeric)
as
$$
select count(tab_nom),  round(avg(extract ('year' from age(birth))),2) from employee;
$$
Language SQL;

select empl_age();
select cnt, ag from empl_age_1();
```

Output Messages Notifications

cnt bigint	ag numeric
10	42.80

Пример пользовательской функции

```
create function y_empl() returns setof varchar
as
$$
select fio from employee where age(birth)<interval'35 years';
$$
language sql;

select * from  y_empl ();
```

Output Messages Notifications

y_empl character varying
Михайлов Е.Н.
Троицкий Т.Т.
Зябликов З.З.
Звонкий С.П.

Пример пользовательской функции

```
create function list_empl(age int) returns table(fio varchar, age interval)
as
$$
select fio, age(birth) from employee where extract('year' from age(birth))<age;
$$
language sql;

select * from  list_empl(35);
```

Output Messages Notifications

fio character varying	age interval
Михайлов Е.Н.	23 years 2 mons 15 days
Троицкий Т.Т.	27 years 12 days
Зябликов З.З.	28 years 9 days
Звонкий С.П.	25 years 9 mons 10 days

Передача значений аргументов

Позиционный способ

- Значения в порядке определения

По имени

- Имя=>*значение*

Смешанный способ

- позиционные перед именными

Примеры пользовательских функций

```
create function my_sum (int, int) returns int
as
$$
select $1+$2;
$$
Language SQL;

select my_sum (1, 2);

create function my_sum (int, int, int) returns int
as
$$
select $1+$2+$3;
$$
Language SQL;

select my_sum (1, 2, 3);
```

Перегрузка пользовательских функций

Перегрузка

- одинаковые имена
- различие в количестве и типах входных аргументов

Перегрузка ≠ REPLACE

- создается новая функция, а не заменяется тело существующей

Пример перегрузки

```
19  create or replace function my_sum (int, int, int default 1) returns int
20  as
21  $$
22  select $1+$2+$3;
23  $$
24  Language SQL;
25
26  select my_sum (1, 2);
27
```

Data Output Messages Notifications

```
ERROR:  function my_sum(integer, integer) is not unique
LINE 1: select my_sum (1, 2);
               ^
HINT:  Could not choose a best candidate function. You might need to add explicit type casts.
SQL state: 42725
Character: 8
```

Оператор case when

Полная форма (любое условие)

```
CASE WHEN условие THEN результат
     [WHEN ...]
     [ELSE результат]
END
```

Упрощенная форма (условие выражение=значение)

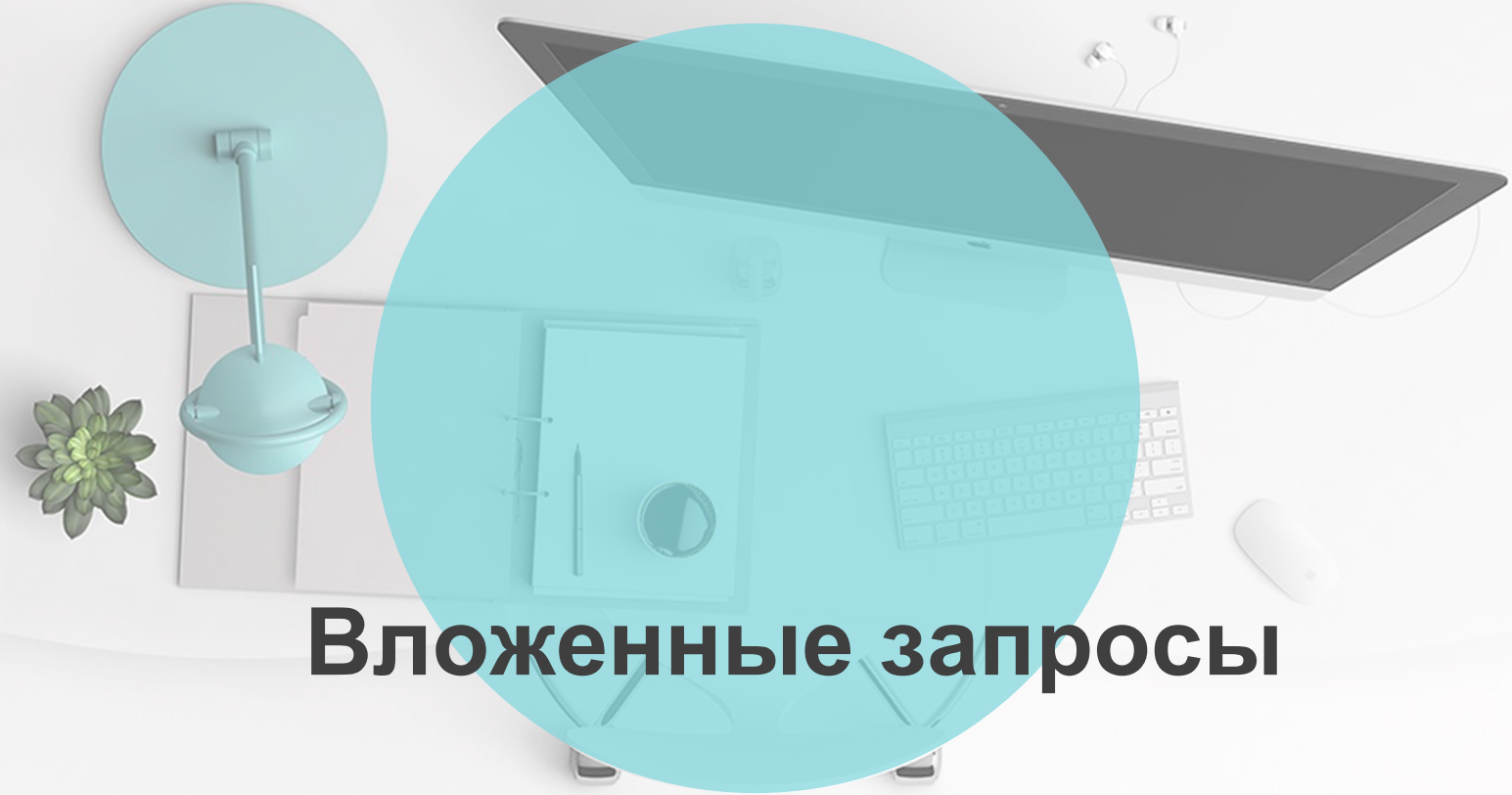
```
CASE выражение
    WHEN значение THEN результат
    [WHEN ...]
    [ELSE результат]
END
```

Пример использования case when

```
--3.Создайте функцию, возвращающую из отделов тот, в котором средний оклад больше. На вход функции подаются названия двух отделов.

--функция расчета среднего оклада в отделе
create function avg_sal (varchar) returns numeric
as
$$
select round(avg(salary),2)
from emp_pos_salary
where name_dep=$1;
$$
Language SQL;

--основная функция
create function dep_salary (dep_1 varchar, dep_2 varchar) returns varchar
as
$$
select case when avg_sal(dep_1)>avg_sal(dep_2) then dep_1 else dep_2 end;
$$
Language SQL;

select * from dep_salary ('отдел маркетинга', 'отдел кадров');
```

Output Messages Notifications

dep_salary character varying
отдел маркетинга



Вложенные запросы

Использование вложенных запросов

- для фильтрации строк (where)
- для фильтрации групп (having)
- в соединении (join)

Скалярные вложенные запросы для фильтрации

Скалярный (SELECT …) возвращает единственное значение(скаляр)

```
SELECT ( выражение [, ...] )
    FROM элемент_FROM
      WHERE имя_столбца оператор (SELECT …)
```

оператор: *= < > >= <= is*

Пример

```
--вывести сотрудников, которые были в командировках менее 3-х раз
select fio, count(bt_id)
from b_trip join employee on emp=tab_nom
group by fio
having count(bt_id)<3;
```

Output Messages Notifications

fio character varying (150)	count bigint
Иванов И.И.	2
Сидоров С.Д.	1
Петров П.П.	1

Вложенные запросы не скалярных типов

Не скалярный (SELECT …) возвращает список значений или таблицу

Выражения подзапросов

- EXISTS (подзапрос)
- выражение IN (подзапрос)
- выражение NOT IN (подзапрос)
- выражение **оператор** ANY (подзапрос)
- выражение **оператор** SOME (подзапрос)
- выражение **оператор** ALL (подзапрос)

Вместо выражение можно использовать конструктор_строки row(столбцы)

Пример

```
--вывести top-3 должностей по уровню окладов
select pos_name, salary from job_position
where salary in
(select salary from job_position order by salary desc limit 3) order by salary desc;
```

Output Messages Notifications

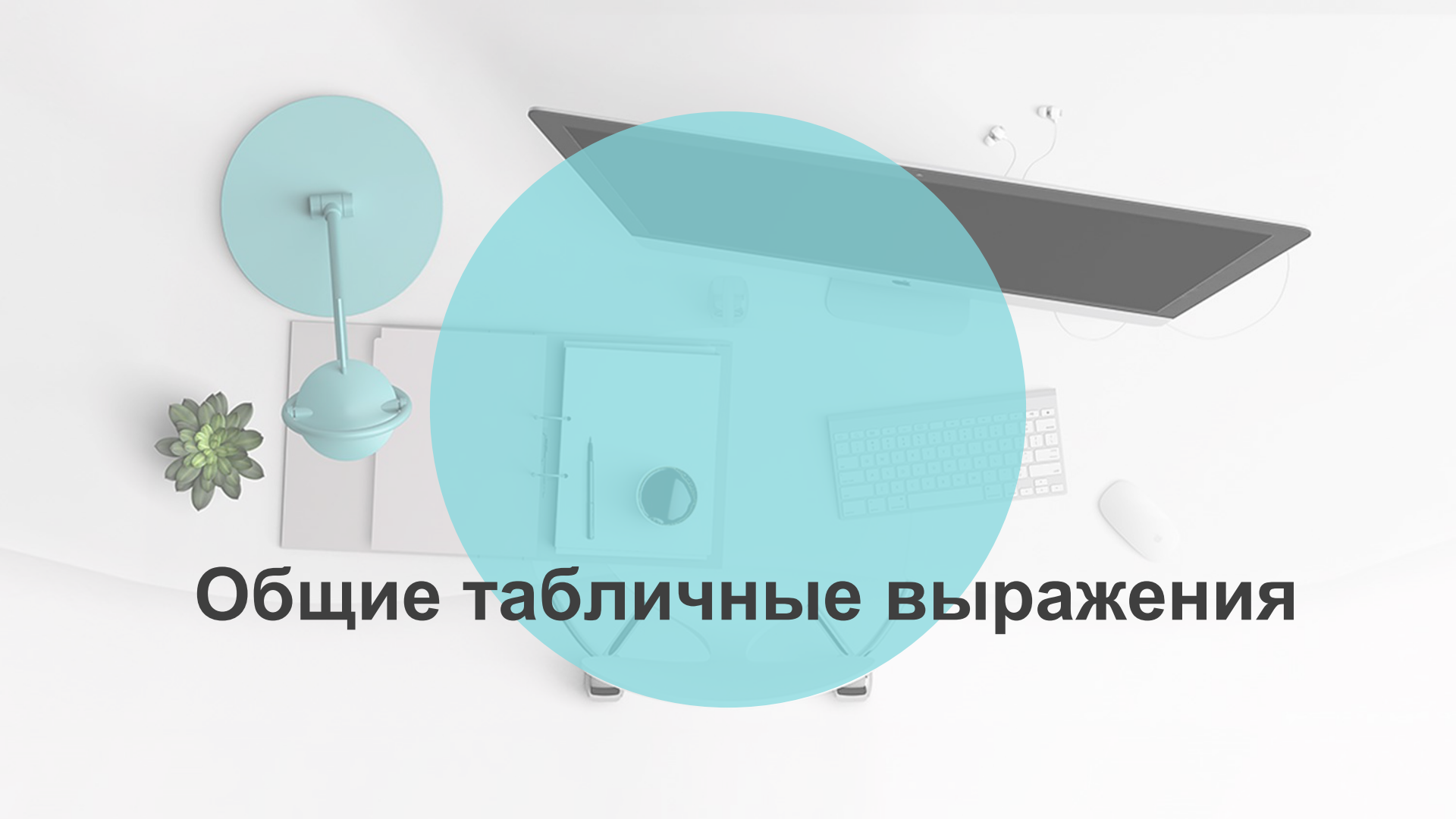
pos_name character varying (100)	salary integer
электрик	70000
главный специалист отдела разработки	60000
ведущий специалист отдела разработки	55000
бухгалтер	55000
системный администратор	55000

Пример

```
select distinct fio, birth
from employee join (select emp from b_trip group by emp having count(bt_id)<3) reserve
on tab_nom = emp;
```

Output Messages Notifications

fio character varying (150)	birth date
Иванов И.И.	1980-02-03
Петров П.П.	1975-01-08
Сидоров С.Д.	1988-02-13



Общие табличные выражения

Понятие общих табличных выражений

Общие табличные выражения (Common Table Expressions, CTE) — это временные именованные результирующие наборы, которые существуют только в рамках выполнения одного SQL-запроса.

```
WITH cte_name AS
( -- подзапрос SELECT ... )
SELECT * FROM cte_name;
```

Пример

```sql
with dep_avg_salary AS (
 -- CTE: средний оклад по департаменту
select dep_name, avg(salary) as avg_salary
from emp_pos_dep
group by dep_name
)
-- основной запрос
select
    fio, dep_name, salary, round(salary - avg_salary,2) as difference
from emp_pos_dep join dep_avg_salary using(dep_name)
order by difference desc;
```

Output Messages Notifications

fio character varying (150)	dep_name character varying (100)	salary integer	difference numeric
Савельева О.С.	отдел кадров	50000	5000.00
Иванов И.И.	отдел маркетинга	50000	5000.00
Смирнов С.М.	отдел разработки	60000	1666.67

Пример каскада CTE

```
WITH regional_sales AS (
    SELECT region, SUM(amount) AS total_sales
    FROM orders
    GROUP BY region
), top_regions AS (
    SELECT region
    FROM regional_sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
)
SELECT region,
       product,
       SUM(quantity) AS product_units,
       SUM(amount) AS product_sales
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;
```